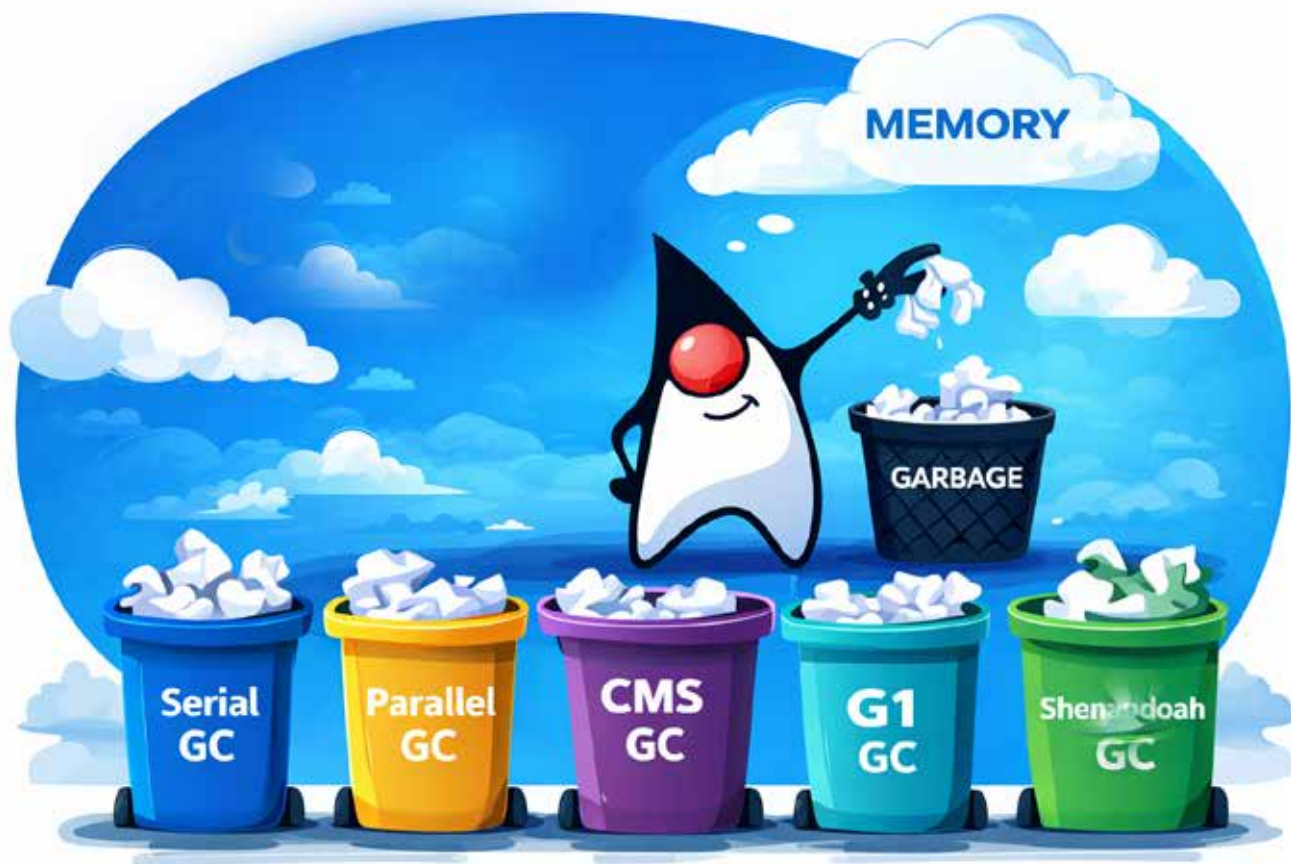


An In-Depth Look at Java Garbage Collectors

Let's dive into the various types of garbage collectors available in Java and lay out a side-by-side comparison to help you choose the right one for your workload.



Garbage collection is an essential pillar of Java's memory management, silently reclaiming unused memory and keeping your application healthy. With each new edition of the Java platform, OpenJDK introduces refinements to how garbage collectors work, aiming for better performance, reduced latency, and improved scalability.

At its core, a garbage collector (GC) frees up memory by automatically removing unused objects, sparing developers the burden of manual memory management. Different applications have different requirements—some demand lightning-fast throughput, others need minimal pause times, and some are designed for massive heap sizes. Java's GC options have grown to accommodate these diverse needs.

A tour of Java's garbage collectors

Let's look at the main types of garbage collectors available in OpenJDK and how they fit into today's Java ecosystem.

Serial GC: Serial GC is the simplest of the bunch, designed for single-threaded environments. It performs all garbage collection tasks in a single thread, making it suitable for small applications or embedded devices where resources are limited. While its implementation is straightforward, its stop-the-world pauses aren't ideal for large-scale or interactive applications.

Parallel GC (Throughput Collector): Parallel GC, sometimes called the Throughput Collector, utilizes multiple threads to speed up the garbage collection process. It's